Agentic workflows

and what is next

Carlos Cotrini

ETH Zurich

21 October 2026

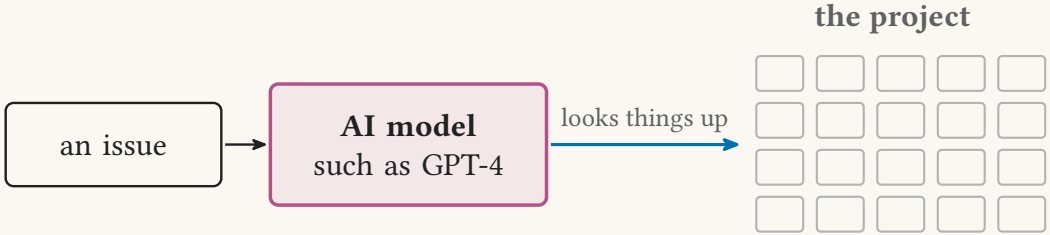
AutoCodeRover

AutoCodeRover: Autonomous Program Improvement

Yuntong Zhang et al., ISSTA 2024

doi.org/10.1145/3650212.3680384

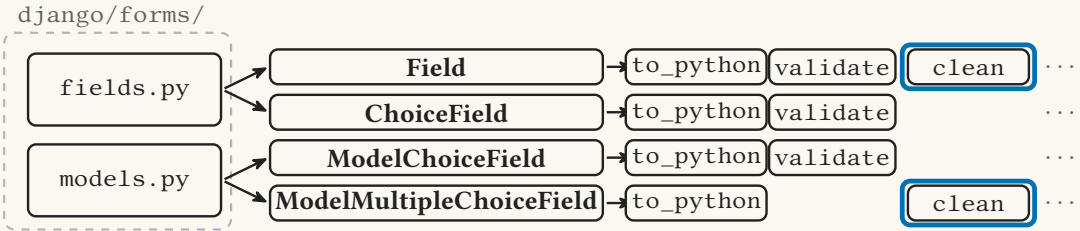
An AI model must search a whole project



Reading everything is too expensive, and searching without structure is hard.

AutoCodeRover gives the project a tree

The paper's idea: give the project the outline it lacks, a tree of its classes and methods.



Asked where clean is defined, the outline answers: Field and ModelMultipleChoiceField.

AutoCodeRover, an AI agent, asks questions of this outline in rounds to find the place to fix, then writes the patch.

Django Software Foundation, *Django 3.2*, PyPI. pypi.org/project/Django/3.2/

Similar score, 6.6 times fewer tokens

The test: 300 real issues from 11 Python projects.

SWE-agent, an agent that works through shell commands: 18.00% fixed



245k tokens, $2.51

AutoCodeRover: 19.00% fixed



37k tokens, $0.43

AutoCodeRover, best of three runs: 26.00% fixed



112k tokens, $1.30

50 of the 57 issues it fixed used under 30k tokens.

Agentless

Agentless: Demystifying LLM-based Software Engineering Agents

Chunqiu Steven Xia et al., FSE 2025

conf.researchr.org/details/fse-2025/fse-2025-research-papers/85

Figures here are those of the author version (32.00%, $0.70); the published version reports 32.67% and $0.68.



Three reasons to doubt the agent

The paper lists what can go wrong when the AI runs the show.

1 Tools are hard

Every tool needs a careful format. A bad one wastes queries.

2 No control

The AI picks the steps.

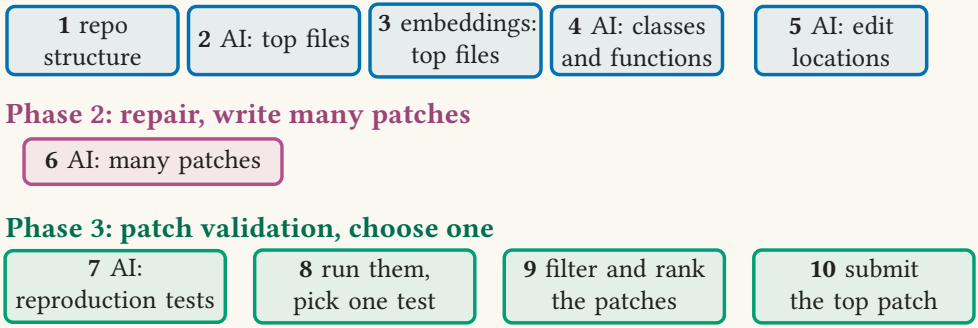
3 Weak self-reflection

LLMs are bad at reviewing themselves.

So the paper asks whether an agent is needed at all.

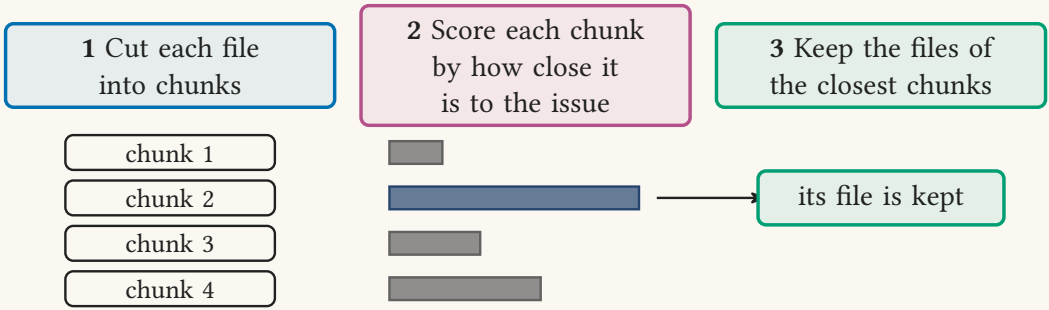
The bet: the program owns the steps

The paper's answer, its Figure 1 redrawn: take every choice away from the AI.



Phase 1: find files with similar code

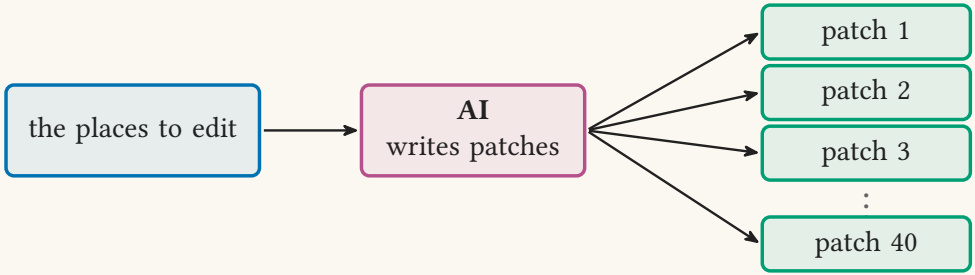
Names can mislead, so search the code too.



Meaning finds files that names miss.

Phase 2: a set of candidate patches

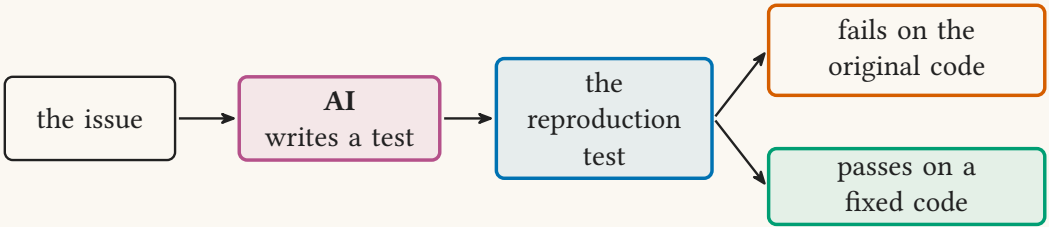
The program asks the AI for many fixes.



The program produces a set of candidate patches, forty per issue.

Phase 3: a test that reproduces the issue

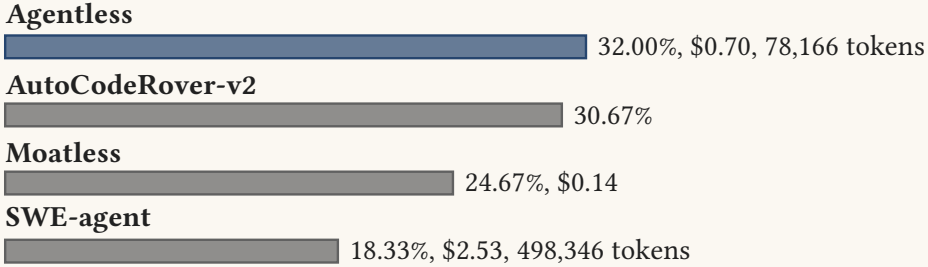
The AI also writes a test of the bug.



The test fails while the bug is there, and passes once the bug is fixed.

Result against agents with the same model

300 real issues from 11 Python projects. All four use GPT-4o.



Fixing every step beats letting the AI search: 32.00% against 30.67%.

Share of issues resolved. Cost per issue. The table gives no cost for AutoCodeRover-v2.

Why context matters

NoLiMa: Long-Context Evaluation Beyond Literal Matching

Ali Modarressi et al., ICML 2025 `proceedings.mlr.press/v267/modarressi25a.html`

Lost in the Middle: How Language Models Use Long Contexts

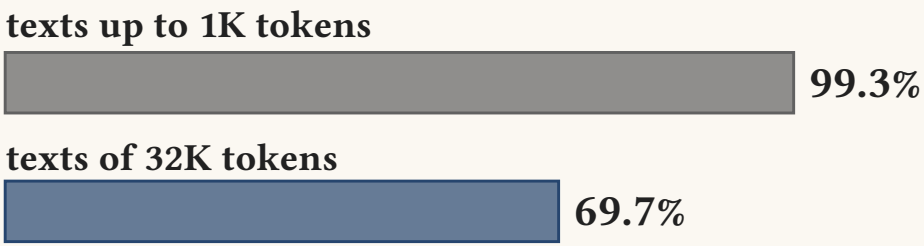
Nelson F. Liu et al., TACL 2024 `aclanthology.org/2024.tacl-1.9`

Slides after Dr. João Carvalho, Uthereal, CAS AI in Industry, ETH Zurich, autumn 2026.

More context, worse answers

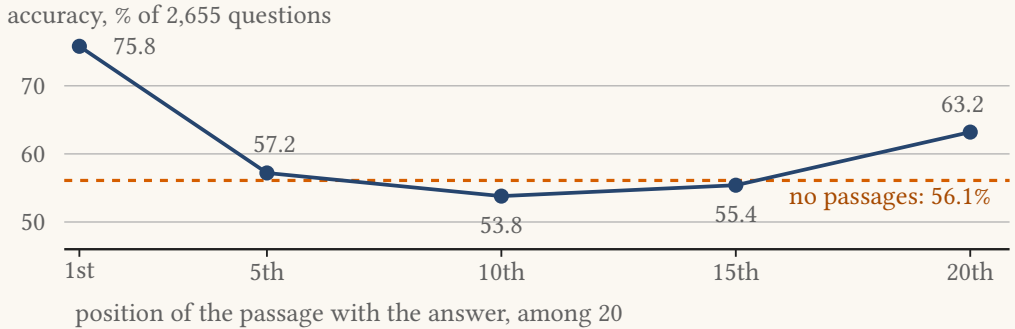
Windows now hold 128K to 1M tokens. A test hides one fact in a long text.

GPT-4o, share of correct answers



The U-curve: where the answer sits matters

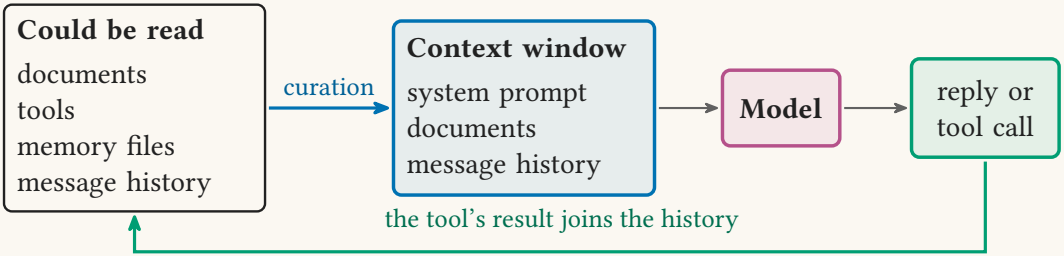
Twenty passages go to GPT-3.5-Turbo, and the one with the answer moves.



Where a passage sits in the window is a harness decision, made before the model is called.

Curating the context window

The window is finite, so something has to choose what goes into it.



In an agent, the harness curates the window again on every call.

Simplified from Anthropic's figure.

Anthropic Engineering, *Effective context engineering for AI agents*, 29 September 2025. `www.anthropic.com/engineering/effective-context-engineering-for-ai-agents`

Context files

On the Use of Agentic Coding Manifests: An Empirical Study of Claude Code

Worawalan Chatlatanagulchai et al., PROFES 2025

doi.org/10.1007/978-3-032-12089-2_40

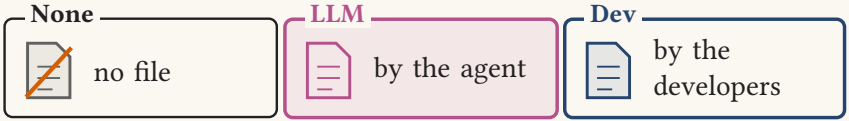
Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?

Thibaud Gloaguen et al., arXiv preprint (v3), ETH Zurich and LogicStar.ai

arxiv.org/abs/2602.11988

The same task, with three kinds of file

An agent reads CLAUDE.md or AGENTS.md first.
The second study changes only that file.



Tasks solved	the baseline	−0.5 **and** −2 **points**	+2.4 **points**
Dollars per task	the baseline	+20% **and** +23%	+19%

Multi-turn conversations

LLMs Get Lost In Multi-Turn Conversation

Philippe Laban et al., ICLR 2026, an Outstanding Paper

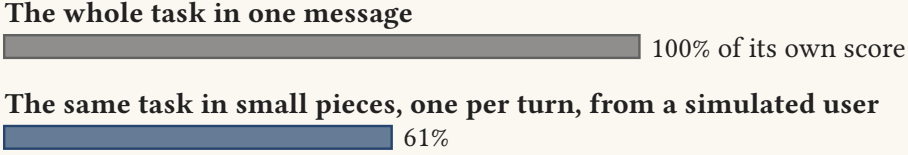
`iclr.cc/virtual/2026/poster/10009146`

Numbers from arXiv v1 (May 2025), the only arXiv version; ICLR's abstract is identical.



All 15 models did worse over several turns

600 tasks of six kinds (code and SQL among them) went to 15 leading models, whole and in pieces.



On average the models scored 39% lower when the task came in pieces.

Over 200,000 simulated conversations; a model, GPT-4o-mini, played the user. Tasks in English; models from 2024 and early 2025.

A-Mem

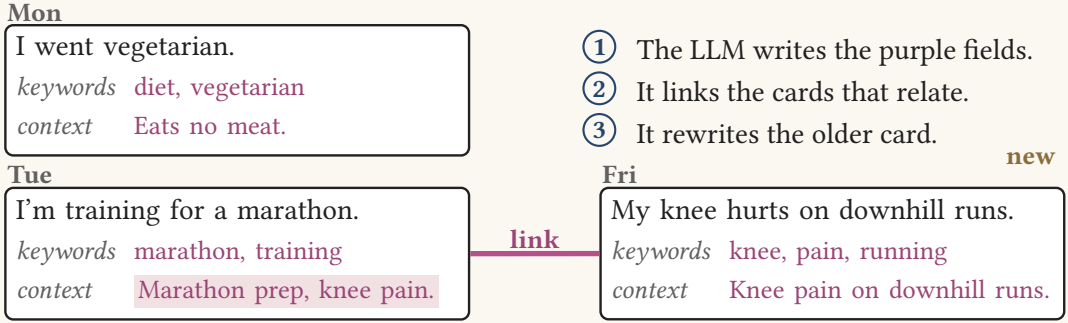
A-Mem: Agentic Memory for LLM Agents

Wujiang Xu et al., NeurIPS 2025

proceedings.neurips.cc/paper_files/paper/2025/hash/19909c36f51abc4856b4560aff3d36d6-Abstract-Conference.html

A memory built like a Zettelkasten

A box of index cards: each memory is a card with keywords and links to related cards.



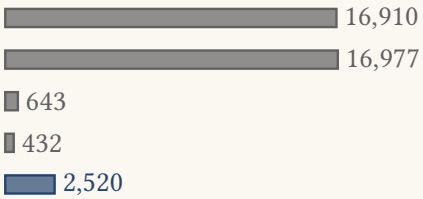
No fixed schema: the LLM decides the keywords, the links and the rewrites.

Toy example: ours; the structure follows Sec. 3 of the paper.

Better answers, far fewer tokens

Questions about long chats (LoCoMo), answered by GPT-4o-mini with each memory.

Memory	Average rank	Tokens per question
The whole chat in the prompt	**2.4**	16,910
MemGPT	**2.4**	16,977
ReadAgent	**4.2**	643
MemoryBank	**4.8**	432
A-Mem	**1.2**	2,520



A-Mem ranks first on average, with 2,520 tokens per question against about 17,000 for the whole chat.

Rank: the average over the five question types by F1, 1 is best. The whole chat still wins on questions that have no answer.

Self-Repair

Is Self-Repair a Silver Bullet for Code Generation?

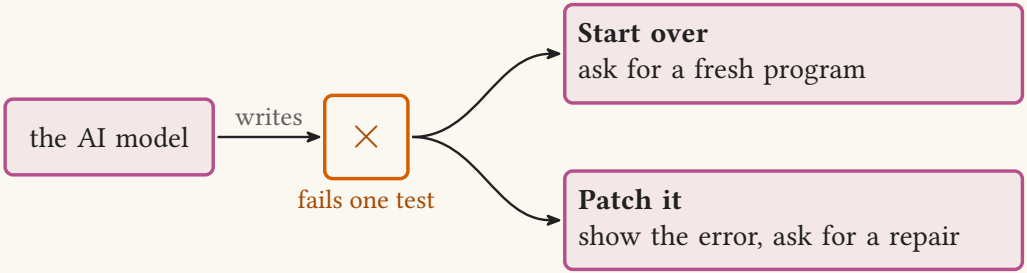
Theo X. Olausson et al., ICLR 2024

iclr.cc/virtual/2024/poster/17429



Patch it or start over

A program fails one test: patch it or start over. An AI model faces the same choice.



One call each, at about the same price: time, money or GPU hours.

Self-repair "requires more invocations of the model", says the paper.

Where repairing pays

At equal cost, repair beat fresh programs in two cases.

GPT-4 on APPS ■ first program □ repair against no repair

10 programs, one repair each



1.05 × repair wins

2 programs, ten repairs each



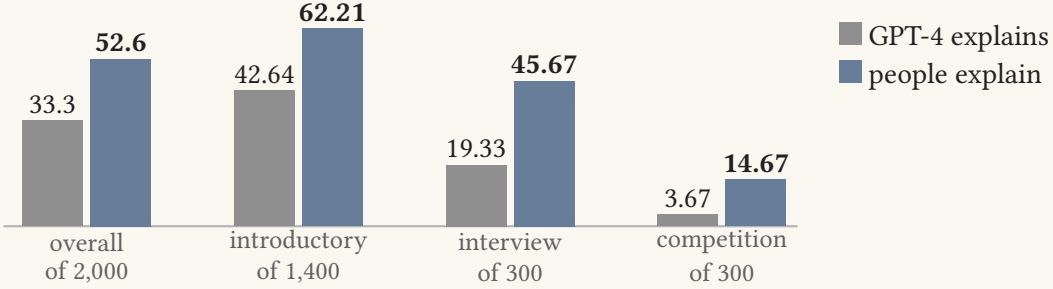
0.97 × repair loses

A stronger model writes the explanation.
Repair then beats fresh programs at every budget.

Repair pays with many first programs and a strong explanation.

People’s explanations repair more often

Sixteen people and GPT-4 each explained 40 failing programs twice; GPT-4 wrote 25 repairs per explanation, 2,000 per source.



Overall 1.58 times as many repairs pass: 52.6% against 33.3%.

Bars: the share of repairs that pass all tests. The authors: the study isolates the explanation. A person’s effort is a cost it leaves unmeasured.